

CampusLink

Guide de l'équipe — état du projet, guide utilisateur et guide technique

Version du 8 octobre 2026 · branche main

Plateforme collaborative pour la vie étudiante : une application web progressive (PWA) installable et utilisable hors ligne, qui regroupe l'emploi du temps, les annonces, les réservations, le forum, le suivi étudiant, le covoiturage, la marketplace de notes et le réseau alumni.

Dépôt	github.com/doniesgh/CampusLink-NextJs (branche <code>main</code>)
Projet	4e année — ESPRIT · d'après le cahier des charges CampusLink v1.0
Public	Membres de l'équipe (développement, tests, démonstration)
Stack	Next.js 16 · React 19 · Express 5 · MongoDB · Socket.IO · Playwright

Sommaire

- 1 Vue d'ensemble du projet
- 2 Modules implémentés
- 3 Ce qui n'est pas encore implémenté
- 4 Guide utilisateur
 - 4.1 Premiers pas · 4.2 Étudiant · 4.3 Enseignant · 4.4 Administration · 4.5 Alumni · 4.6 Comptes de démonstration
- 5 Faire tourner le projet (guide de l'équipe)
 - 5.1 Prérequis · 5.2 Après un git pull · 5.3 Installation · 5.4 Variables · 5.5 Scripts et ports · 5.6 Mode production local
- 6 Tests
- 7 Organisation du code et règles de travail
- 8 Déploiement gratuit (Vercel + Render + MongoDB Atlas)
- 9 Sécurité : ce que chacun doit savoir
- 10 Problèmes fréquents et solutions
- 11 Documentation de référence

1. Vue d'ensemble du projet

CampusLink centralise les services numériques de la vie étudiante dans une seule application, installable sur téléphone et ordinateur sans passer par un store, et consultable hors ligne pour l'essentiel (emploi du temps, annonces). Une application mobile **Flutter** est prévue : elle utilisera la même API.

Architecture en bref

Partie	Rôle	Dossier
Application web	Next.js 16 (App Router), React 19, Tailwind CSS 4, next-intl (français / anglais). Pages serveur, Server Actions, relais <code>/bff</code> vers l'API, service worker (hors ligne, push).	<code>next/</code>
API	Express 5 + Mongoose 9 : routes REST <code>/api/*</code> , temps réel Socket.IO (chat), tâches planifiées, e-mails, push, PDF.	<code>backend/</code>
Base de données	MongoDB 7 (local, Docker ou MongoDB Atlas).	—
Tests	Playwright : tests d'API, tests de sécurité, tests navigateur.	<code>tests/</code>
Documentation	Architecture, déploiement, spécifications, contrats d'API, qualité, démos.	<code>docs/</code>

Principe clé. Le navigateur ne parle jamais directement à l'API REST : les jetons de session sont dans des cookies `httpOnly` gérés par Next.js, qui appelle l'API avec un jeton `Bearer`. Seul le chat temps réel se connecte directement à l'API, avec un ticket de 60 secondes. L'API ne dépend d'aucun cookie, ce qui la rend directement utilisable par l'application Flutter.

Rôles

Rôle	Ce qu'il peut faire
Étudiant	Emploi du temps de son groupe, annonces, notifications, réservations, forum, « Mon suivi » (présences, notes, bilan PDF), covoiturage, marketplace, demandes de mentorat.
Enseignant	Son emploi du temps, annonces vers ses groupes, appel, notes, réservations, forum (réponses certifiées), marketplace, réseau alumni.
Administration	Tout gérer : utilisateurs, structure académique, emploi du temps, annonces, ressources et réservations, modération (forum, marketplace, alumni), suivi des étudiants, journal d'audit.
Alumni	Annonces, forum, profil alumni (avec consentement), mentorat, fil d'actualités alumni.

L'inscription publique crée toujours un compte étudiant. Les autres rôles sont attribués par l'administration.

2. Modules implémentés

Les dix modules du périmètre de base du cahier des charges sont en place (le module 10 partiellement).

#	Module	Fonctionnalités livrées	État
1	Emploi du temps intelligent	Saisie par l'administration (séances, séries hebdomadaires) ou import CSV ; détection des conflits (salle, enseignant, groupe) ; vues jour / semaine / mois ; filtres matière et enseignant ; changements de salle ou d'horaire et annulations mis en évidence et notifiés (in-app + push) ; export iCal / Google Agenda ; consultable hors ligne.	Fait
2	Covoiturage étudiant	Trajets vers ou depuis le campus ; recherche par quartier ou par position (rayon, date, sens) ; coût partagé suggéré ; demandes de place sans surréservation ; chat en temps réel entre conducteur et passagers acceptés ; notes après le trajet ; historique ; coordonnées précises visibles seulement par les participants.	Fait
3	Marketplace de notes	Dépôt de documents (PDF, images, Word, PowerPoint) avec matière, niveau, professeur, année ; validation par l'administration ; signalements ; jetons fictifs (portefeuille, achats sans double débit) ; documents gratuits ou premium ; avis ; recherche et filtres.	Fait
4	Forum d'entraide	Questions par matière, chapitre, niveau et étiquettes ; recherche plein texte et questions similaires ; votes ; réponse acceptée ; réponses certifiées des enseignants ; réputation et badges ; suivi et notifications ; réponses hors ligne ; modération.	Fait
5	Réservation de salles et de matériel	Disponibilités par ressource ; salles libres ; conflits bloqués (y compris avec les cours) ; validation optionnelle ; rappel 60 min avant ; historique ; statistiques d'occupation ; gestion du matériel.	Fait
6	Annuaire et réseau alumni	Profils avec consentement explicite (RGPD) ; annuaire filtrable ; demandes de mentorat (coordonnées partagées seulement après acceptation) ; fil d'actualités ; export et effacement des données.	Fait
7	Notifications et annonces	Annonces avec priorité et pièces jointes ; ciblage par rôle, filière, niveau, groupe ; nombre de destinataires en direct ; publication immédiate ou programmée ; notification in-app + push ; statistiques de lecture.	Fait
8	Mode hors ligne (PWA)	Application installable ; pages et données des modules essentiels gardées hors ligne ; actions hors ligne rejouées au retour du réseau ; bandeau « Tu es hors ligne » ; données effacées à la déconnexion (ordinateurs partagés).	Fait
9	Tableau de bord étudiant	Appel par l'enseignant ; taux d'absence et alertes (10 % / 20 %) ; évaluations et notes publiées ; moyennes pondérées ; graphiques de progression et d'activité ; comparaison anonymisée avec le groupe ; bilan PDF.	Fait
10	Authentification et rôles	Rôles et permissions par module ; jetons JWT + rafraîchissement ; validation en deux étapes par e-mail ; mot de passe oublié ; journal d'audit des actions sensibles.	Partiel

Transversal

- **Langues** : français (par défaut, tutoiement) et anglais, sélecteur dans le menu et sur la page de connexion.

- **Sécurité** : 53 tests de sécurité automatisés (autorisations, accès aux données d'autrui, concurrence, injections, fuites de données, limitation des tentatives, temps réel) ; en-têtes CSP et HSTS.
- **Qualité** : audit Lighthouse sur les pages principales — accessibilité 100, bonnes pratiques 100, performance 91 à 95, premier affichage en 0,8–0,9 s (4G simulée).
- **Données de démonstration** : une commande (`npm run seed:demo`) remplit tous les modules.

3. Ce qui n'est pas encore implémenté

Module 10 — à compléter

- Connexion unique de l'établissement (**SSO** via SAML / OAuth2) : le bouton « Continuer avec le compte de l'école » est affiché mais désactivé (« Bientôt »).
- Connexion **Google** (OAuth).

Modules complémentaires 11 à 18 (phase suivante du cahier des charges)

#	Module	Contenu attendu	Complexité
11	Gamification et badges	Points d'expérience sur toute la plateforme, badges thématiques, classement mensuel par filière. <i>Le forum a déjà une réputation et des badges.</i>	Faible
12	Assistant virtuel / FAQ	Chatbot pour les questions administratives, base de connaissances alimentée par la scolarité, renvoi vers un humain.	Élevée
13	Espace bien-être	Rendez-vous avec le service d'accompagnement, ressources, contact confidentiel ; données sensibles (chiffrement, RGPD strict).	Moyenne à élevée
14	Sondages et votes	Sondages de l'administration ou des associations, vote unique vérifié, résultats en direct ou différés.	Faible
15	Stages et opportunités	Offres des entreprises et alumni, dépôt de CV, candidature, mise en relation avec les alumni. <i>Le fil alumni a déjà un type « opportunité ».</i>	Moyenne
16	Bibliothèque numérique	Supports officiels des enseignants, annales corrigées, droits de diffusion.	Moyenne
17	Clubs et événements	Fiches clubs, calendrier des événements, inscriptions et places, rappels.	Moyenne
18	Objets trouvés et signalements	Annonces objet perdu / trouvé avec photo, signalement d'incident de sécurité avec suivi.	Faible

Autres travaux restants

Sujet	Détail
Application mobile Flutter	Non commencée. L'API est prête (jetons en JSON, codes d'erreur stables, temps réel avec le jeton d'accès).
Push vers mobile (FCM)	Les abonnements FCM sont acceptés et stockés, mais l'envoi n'est pas implémenté.
E-mails de la version en ligne	Le site est en ligne (section 8), mais le SMTP n'y est pas configuré : la connexion avec code par e-mail (2FA) et la réinitialisation du mot de passe ne fonctionnent pas encore en ligne (ne pas activer la 2FA sur un compte en ligne : il ne pourrait plus se connecter). Ajouter <code>SMTP_USER</code> , <code>SMTP_PASS</code> et <code>SMTP_FROM</code> sur Render.

Sujet	Détail
Livrables du cahier des charges	Spécifications fonctionnelles des modules 2, 3 et 6 ; maquettes UI/UX ; tests fonctionnels des phases 2 et 3 (seuls les tests de sécurité existent) ; vidéos de démonstration des phases 2 et 3.
Petites limites connues	Pas de formulaire web pour modifier un trajet (annuler puis reproposer) ; annulation d'un trajet par l'administration seulement via l'API ; chat temps réel sur une seule instance de serveur ; stockage objet S3 et file de messages (RabbitMQ / BullMQ) remplacés par des solutions plus simples.

4. Guide utilisateur

4.1 Premiers pas (tous les rôles)

Se connecter

Ouvrir le site, puis **Se connecter** avec son e-mail et son mot de passe. Cocher « Rester connecté » pour une session de 30 jours. Si la validation en deux étapes est activée, saisir le code à 6 chiffres reçu par e-mail.

Mot de passe oublié : lien sur la page de connexion, puis lien reçu par e-mail (valable 30 min).

Langue

Sélecteur **Langue** en bas du menu (ou sur la page de connexion) : Français / English. Le choix est enregistré sur le compte et utilisé pour les e-mails, les notifications et le bilan PDF.

Mon compte

Menu **Compte** : prénom et nom, langue, **validation en deux étapes**, changement de mot de passe, **notifications push** sur cet appareil (« Activer les notifications push »).

Installer l'application

Sur un site en HTTPS, le navigateur propose « Installer » (Chrome / Edge : icône dans la barre d'adresse ; Android : « Ajouter à l'écran d'accueil » ; iPhone : Partager → « Sur l'écran d'accueil »).

Hors ligne

Les pages déjà consultées (accueil, emploi du temps, annonces, notifications, forum, réservations, suivi) restent lisibles sans réseau. Un bandeau « Tu es hors ligne » s'affiche ; les actions faites hors ligne (annonce lue, réponse au forum) sont envoyées au retour du réseau.

Notifications

La cloche du menu indique les notifications non lues : changements d'emploi du temps, annonces, réservations, forum, présences, notes, covoiturage, marketplace, mentorat. **Tout marquer comme lu** depuis la page Notifications.

4.2 Étudiant

Menu	Ce que tu peux faire
Accueil	Tes cours du jour, les dernières annonces et tes notifications non lues.
Emploi du temps	Vues <i>Jour, Semaine, Mois</i> ; navigation Précédent / Aujourd'hui / Suivant ; filtres <i>Matière et Enseignant</i> . Les changements apparaissent en évidence (« Déplacé depuis C201 », « Annulé »). Exporter : copier le lien du calendrier, télécharger le .ics ou ajouter à Google Agenda.
Annonces	Fil des annonces qui te concernent (priorité Urgente / Importante / Normale / Faible, « Nouveau » si non lue), filtre « Non lues seulement », pièces jointes téléchargeables.
Réservations	Choisir une salle ou du matériel, voir ses disponibilités, trouver les salles libres sur un créneau, réserver (15 min de grille, 7 h–21 h, 3 h max, 3 réservations à venir max), annuler. Certaines ressources (amphis...) demandent une validation.

Menu	Ce que tu peux faire
Forum	Rechercher, filtrer (matière, niveau, étiquette, sans réponse), poser une question (des questions similaires sont proposées), répondre, voter, accepter une réponse à ta question, suivre une question, signaler. Ta réputation et tes badges sont sur ton profil du forum.
Mon suivi	Taux de présence et alertes par matière, moyennes et notes publiées, graphique de progression, activité ; « Me comparer à mon groupe » (anonyme) ; Télécharger mon relevé (PDF) .
Covoiturage	<i>Trouver un trajet</i> (quartier ou « Utiliser ma position », rayon, date, sens), demander une place ; <i>Proposer un trajet</i> (le coût partagé est suggéré) ; <i>Mes trajets</i> . Sur un trajet accepté : chat en direct avec le conducteur et les passagers, puis note après le trajet.
Marketplace	Parcourir et filtrer les documents, télécharger les gratuits, acheter les premium avec tes jetons (100 jetons de départ), déposer tes propres notes (validées par l'administration), laisser un avis, voir ton portefeuille.
Réseau alumni	Annuaire des diplômés (filière, promotion, secteur, « mentorat disponible »), fil d'actualités, demande de mentorat (3 en attente max) — l'e-mail de l'alumni n'est partagé qu'après acceptation.

4.3 Enseignant

Menu	Ce que vous pouvez faire
Emploi du temps	Les séances que vous assurez, avec les mêmes vues, filtres et exports que les étudiants.
Gestion des annonces	Nouvelle annonce : titre, message, priorité, destinataires (uniquement les groupes où vous enseignez cette année, nombre de destinataires affiché), pièces jointes, publication immédiate ou programmée. Statistiques de lecture.
Présences	Faire l'appel d'une séance (de 15 min avant à 7 jours après) : Présent / Absent / En retard, « Tous présents ». Les absences non excusées alimentent les alertes des étudiants.
Notes	Créer une évaluation (type, date, barème, coefficient) pour une matière et un groupe que vous enseignez, saisir les notes, publier (les étudiants sont notifiés). Une évaluation publiée ne peut être supprimée que par l'administration.
Réservations, forum, marketplace, réseau alumni	Comme les étudiants ; au forum, vos réponses portent le badge « certifiée » ; réservations jusqu'à 8 h.

4.4 Administration

Menu Administration	Ce que vous pouvez faire
Utilisateurs	Rechercher, filtrer par rôle, Ajouter un utilisateur (rôle, groupe pour un étudiant), modifier, supprimer.
Structure académique	Onglets Filières, Groupes, Matières, Salles (réservable, validation requise). Un élément encore utilisé ne peut pas être supprimé.
Gestion de l'emploi du temps	Afficher par groupe, enseignant ou salle ; Ajouter une séance (option « Répéter chaque semaine ») ; cliquer une séance pour la modifier, l'annuler (étudiants notifiés) ou la supprimer ; Importer un CSV (« Vérifier le fichier » puis « Importer »). Les conflits sont refusés avec le détail.

Menu Administration	Ce que vous pouvez faire
Gestion des annonces	Toutes les annonces, ciblage libre (rôles, filières, niveaux, groupes), brouillons, programmation, statistiques.
Réservations et ressources	Demandes à valider (accepter / refuser avec note), toutes les réservations, matériel, statistiques d'occupation.
Modération du forum	Signalements, masquer / réafficher un contenu, traiter un signalement.
Modération marketplace	Documents à valider (accepter / refuser avec motif), signalements, documents retirés.
Suivi des étudiants	Alertes d'absentéisme, vue par groupe, fiche d'un étudiant et son bilan PDF ; « Excusé » sur les absences justifiées.
Présences, Notes	Comme un enseignant, pour toutes les séances et tous les groupes.
Journal d'audit	Qui a fait quoi et quand (création de comptes, changements de rôle, séances, annonces, décisions, modération...), filtrable par action.

Format du fichier CSV d'emploi du temps

```
date,start,end,subject_code,teacher_email,groups,room,type,notes
2026-10-12,08:30,10:00,BDD,amira.bensalah@campuslink.local,4TWIN1|4TWIN2,A04,LECTURE,
2026-10-12,10:15,11:45,WEB,karim.trabelsi@campuslink.local,4TWIN1,C201,LAB,Apporter son PC
```

Séparateur virgule ou point-virgule, UTF-8, heures du campus, groupes séparés par « | », types LECTURE, TUTORIAL, LAB, EXAM, OTHER. Tout ou rien : une ligne invalide ou en conflit bloque l'import (le rapport indique le numéro de ligne).

4.5 Alumni

- **Mon profil alumni** : parcours, entreprise, poste, secteur, compétences, disponibilité pour le mentorat. Le profil reste **privé** tant que vous n'avez pas donné votre consentement explicite à sa visibilité sur le campus.
- **Mentorat** : accepter ou refuser les demandes des étudiants (avec un message) ; votre e-mail n'est partagé qu'après acceptation.
- **Fil d'actualités** : partager un nouveau poste, une réussite, une opportunité, un événement.
- **Mes données** : « Télécharger mes données » (export RGPD) ou les supprimer définitivement.

4.6 Comptes de démonstration

Créés par `npm run seed:demo`. **Mot de passe commun : Campus123!** (à ne jamais utiliser en production).

Rôle	Comptes (e-mail)
Administration	<code>admin@campuslink.local</code>
Enseignants	<code>amira.bensalah@...</code> (Bases de données), <code>karim.trabelsi@...</code> (Développement web), <code>leila.gharbi@...</code> , <code>mehdi.jaziri@...</code>
Étudiants 4TWIN1	<code>yasmine.haddad@...</code> (alerte d'absence en WEB), <code>sarra.mansouri@...</code> (alerte critique en BDD), <code>omar.ferchichi@...</code> (interface en anglais), <code>mariem.jlassi@...</code> , <code>aziz.benamor@...</code>

Rôle	Comptes (e-mail)
Autres groupes	4TWIN2 : ahmed.karray@..., nour.chaabane@..., rami.belhaj@... · 4DS1 : ines.zouari@..., youssef.ammami@..., malek.ayari@... · 4SAE1 : aya.khelifi@..., hamza.dridi@..., lina.bouazizi@... · sans groupe : skander.mejri@...
Alumni	selim.rekik@..., bilel.saidi@..., emna.chebbi@..., firas.guesmi@..., hela.tlili@..., mouna.kacem@..., rania.gharsalli@..., walid.mejdoub@...

« @... » = @campuslink.local . Les données (emploi du temps de la semaine, annonces, réservations, forum, notes, trajets, documents, alumni) sont générées autour de la date du jour.

5. Faire tourner le projet

5.1 Prérequis

Outil	Version	Remarque
Node.js	20 ou plus (testé : 22)	Inclut npm. Utiliser npm (pas pnpm).
MongoDB	7 ou plus	Service local, conteneur Docker ou MongoDB Atlas.
Git	récent	—
Docker Desktop	facultatif	Le plus simple pour lancer MongoDB.
Navigateur	Chrome / Edge / Firefox récents	Pour la PWA et le hors ligne en production.

5.2 À faire après le premier `git pull` de cette version

Important. `backend/.env` n'est plus suivi par git (il contenait des secrets). **Sauvegardez votre `backend/.env` avant de faire `git pull`** : git le supprime de votre dossier. Recréez-le ensuite à partir de `backend/.env.example`. L'ancienne variable `SECRET` n'est plus lue : `JWT_SECRET` est obligatoire.

1. Sauvegarder `backend/.env`, puis `git pull` sur `main`.
2. `npm install` dans `backend/`, `next/` et `tests/` (nouvelles dépendances).
3. Recréer `backend/.env` depuis `backend/.env.example` (étape 5.3).
4. Remplir la base : `npm run seed:demo`, puis créer son compte administrateur.

5.3 Installation pas à pas

1. Récupérer le code et lancer MongoDB

```
git clone https://github.com/doniesgh/CampusLink-NextJs.git
cd CampusLink-NextJs
# MongoDB dans Docker (si vous n'avez pas de service MongoDB local)
docker run -d --name campuslink-mongo -p 27017:27017 -v campuslink-data:/data/db mongo:7
```

2. API (backend)

```
cd backend
npm install
cp .env.example .env # Windows PowerShell : Copy-Item .env.example .env
# Générer un secret et le coller dans JWT_SECRET= (fichier .env)
node -e "console.log(require('crypto').randomBytes(64).toString('hex'))"
npm run generate-vapid # copier les deux clés dans VAPID_PUBLIC_KEY /
VAPID_PRIVATE_KEY
npm run seed:demo # données de démonstration (facultatif)
npm run create-admin -- votre.email@esprit.tn MotDePasseSolide Prénom Nom
npm run dev # API sur http://localhost:4000 (redémarre à chaque
modification)
```

3. Application web (autre terminal)

```
cd next
npm install
cp .env.example .env.local          # API_URL=http://localhost:4000 par défaut
npm run dev                         # site sur http://localhost:3000
```

E-mails en local. Sans `SMTP_USER` / `SMTP_PASS`, les e-mails (code de connexion, lien de réinitialisation) ne sont pas envoyés : ils s'affichent dans la console de l'API. Pour un vrai envoi, utiliser un mot de passe d'application Gmail.

5.4 Variables d'environnement essentielles

Variable	Où	Rôle
<code>MONGO_URI</code>	API	Connexion MongoDB (obligatoire), ex. <code>mongodb://localhost:27017/campuslink</code>
<code>JWT_SECRET</code>	API	Secret de signature des jetons (obligatoire, 64 octets aléatoires)
<code>CORS_ORIGINS</code> , <code>APP_URL</code>	API	Adresse du site (CORS, liens des e-mails) — <code>http://localhost:3000</code> en local
<code>PUBLIC_API_URL</code>	API	Adresse publique de l'API (liens iCal) — <code>http://localhost:4000</code>
<code>VAPID_PUBLIC_KEY</code> , <code>VAPID_PRIVATE_KEY</code>	API	Notifications push (<code>npm run generate-vapid</code>)
<code>SMTP_*</code>	API	Envoi réel des e-mails (facultatif en local)
<code>RATE_LIMIT_ENABLED</code>	API	<code>false</code> pour désactiver la limitation des tentatives pendant le développement
<code>STORAGE_DIR</code> / <code>STORAGE_DRIVER</code>	API	Fichiers envoyés : sur disque (<code>disk</code>) ou dans MongoDB (<code>gridfs</code> , hébergement sans disque)
<code>API_URL</code>	Web	Adresse de l'API vue par le serveur Next.js
<code>NEXT_PUBLIC_REALTIME_URL</code>	Web	Adresse de l'API pour le chat temps réel (lue au build)
<code>NEXT_PUBLIC_APP_TIMEZONE</code>	Web	Fuseau du campus (<code>Africa/Tunis</code>)
<code>COOKIE_SECURE</code>	Web	<code>false</code> seulement pour un build de production servi en http local

La liste complète et commentée est dans `backend/.env.example` et `next/.env.example`.

5.5 Scripts et ports

Commande	Effet	Port	Utilisé par
backend/		3000	Site (Next.js)
<code>npm run dev</code>	API avec rechargement (nodemon)	4000	API + temps réel (Socket.IO)

Commande	Effet	Port	Utilisé par
<code>npm start</code>	API (production)	27017	MongoDB
<code>npm run seed:demo</code>	Données de démonstration	27018	Tests : MongoDB en mémoire
<code>npm run create-admin - e-mail mdp</code>	Compte administrateur	4100 , 4101	Tests : API (et API avec limitation)
<code>npm run generate-vapid</code>	Clés push	3100	Tests : site
next/			
<code>npm run dev</code>	Site en développement	Next.js 16 n'accepte qu'un seul <code>next dev</code> par dossier : arrêtez votre <code>npm run dev</code> avant les tests navigateur, ou lancez-les avec <code>E2E_NEXT_MODE=start</code> .	
<code>npm run build / npm start</code>	Build et serveur de production		
<code>npm run lint</code>	ESLint		
tests/			
<code>npm run test:api</code>	Tests d'API et de sécurité		
<code>npm run test:e2e</code>	Tests navigateur		

5.6 Mode production local (PWA, hors ligne, push)

Le service worker (hors ligne, notifications push) n'est actif qu'en build de production :

```
cd next
npm run build
COOKIE_SECURE=false npm start # PowerShell : $env:COOKIE_SECURE="false"; npm start
```

Ensuite : DevTools → Application pour voir le service worker et le manifeste, DevTools → Network → *Offline* pour tester le hors ligne. L'installation de l'application et le push fonctionnent sur `localhost` ou en HTTPS.

6. Tests

Les tests Playwright démarrent **leur propre** base MongoDB en mémoire, leur API et leur site sur des ports dédiés : ils ne touchent jamais à vos données locales.

```
cd tests
npm install
npx playwright install chromium      # une seule fois
npm run test:api                     # API + sécurité (≈ 2 min)
npm run test:e2e                     # navigateur (≈ 2 min)
npx playwright test --project=api api/security.spec.ts # sécurité seulement
npm run report                       # rapport HTML du dernier passage
```

Ensemble	Tests	Contenu
API	47	Authentification, sessions, comptes, profils, mots de passe, administration.
Sécurité (<code>security.spec.ts</code>)	53	Autorisations par rôle, accès aux données d'autrui, affectation de champs interdits, concurrence (réservations, places, portefeuille), fichiers, injections, fuites, limitation, temps réel, protections du site.
Navigateur (e2e)	22	Accueil, inscription, connexion, sessions, validation en deux étapes, mot de passe, hors ligne (confidentialité sur ordinateur partagé).

Dernier passage complet sur un build de production : **122 / 122 réussis**.

7. Organisation du code et règles de travail

```
backend/          API Express
  app.js          démarrage, middlewares, routes, temps réel
  models/ controllers/ routes/ service/ middleware/ utils/
  scripts/        create-admin, generate-vapid, seed-demo (+ seed/10...80, un fichier par
module)
  docs/           documentation de chaque module (bookings, forum, analytics, carpool,
marketplace, alumni)
next/            application web
  app/            pages : (front) accueil, (auth) connexion, (back)/dashboard/..., bff/,
offline/
  components/     interface : ui/ (composants communs), shell/ (menu), un dossier par module
  lib/            logique : appels API, sessions, hors ligne, temps réel, un dossier par
module
  messages/fr|en/ textes traduits, un fichier par module
  public/sw.js    service worker
  proxy.ts        protection des pages, rafraîchissement des jetons, CSP
tests/           Playwright (api/, e2e/, support/)
docs/            architecture, déploiement, spécifications, contrats, qualité, démos, ce
guide
```

Règles à respecter

- **Contrats** : chaque phase a un contrat (docs/phase1|2|3-contract.md) qui fixe les routes, les règles, les codes d'erreur et les écrans. Modifier le contrat avant de changer un comportement.
- **Erreurs de l'API** : toujours { error, code, details? } avec un code stable (le web et Flutter s'appuient dessus).
- **Textes** : jamais de texte en dur dans l'interface ; ajouter la clé dans messages/fr/...json **et** messages/en/...json (vérifié par TypeScript). Français au tutoiement.
- **Appels API côté web** : par le serveur (serverApi , Server Actions) ou par le relais /bff ; jamais d'appel direct du navigateur vers l'API REST.
- **Couleurs** : utiliser les jetons du thème (bg-primary , text-muted-foreground ...), jamais de couleur en dur (mode sombre).
- **Avant de pousser** : npx tsc --noEmit et npm run lint dans next/, npm run test:api dans tests/.
- **Git** : une branche par fonctionnalité, des commits courts et thématiques, une pull request vers main . Ne jamais committer .env .

8. Déploiement gratuit (Vercel + Render + MongoDB Atlas)

Configuration choisie pour une mise en ligne gratuite : le site sur **Vercel**, l'API sur **Render**, la base sur **MongoDB Atlas**. HTTPS est fourni par les deux hébergeurs (nécessaire pour la PWA et le push).

Étapes

1. **MongoDB Atlas** (atlas.mongodb.com) : créer un cluster gratuit M0, un utilisateur de base de données, autoriser l'accès réseau 0.0.0.0/0 (Render n'a pas d'IP fixe), copier la chaîne de connexion `mongodb+srv://...`.
2. **Render** (render.com) : *New* → *Blueprint*, choisir le dépôt GitHub ; le fichier `render.yaml` du dépôt crée l'API (dossier `backend`, `node app.js`, contrôle de santé `/api/health`, fichiers stockés dans MongoDB). Renseigner `MONGO_URI`, les clés VAPID, l'adresse du site, et le SMTP si besoin.
3. **Vercel** (vercel.com) : *Add New* → *Project*, importer le dépôt, *Root Directory* = `next`, variables `API_URL` et `NEXT_PUBLIC_REALTIME_URL` = adresse Render, `NEXT_PUBLIC_APP_TIMEZONE` = `Africa/Tunis`, `TRUSTED_PROXY_HOPS` = `1`. Le fichier `next/vercel.json` impose le framework Next.js et `npm ci` (sinon Vercel sert le dossier comme un site statique, ou choisit `pnpm` à cause de l'ancien `pnpm-lock.yaml`).
4. Reporter l'adresse Vercel dans `CORS_ORIGINS` et `APP_URL` sur Render.
5. Remplir la base en ligne depuis un poste de l'équipe : `MONGO_URI=<Atlas> STORAGE_DRIVER=gridfs npm run seed:demo`, puis `npm run create-admin`.
6. Garder l'API éveillée : un moniteur gratuit (ex. UptimeRobot) qui appelle `/api/health` toutes les 10 minutes.

Limites de l'offre gratuite. Sans requête pendant 15 minutes, l'API Render se met en veille (premier appel lent, ≈ 50 s) — d'où le moniteur. Atlas gratuit : 512 Mo (données + fichiers). Le chat et les tâches planifiées tournent sur une seule instance.

Version en ligne

Élément	Adresse / détail
Site web (PWA)	<code>https://campuslink-esprit.vercel.app</code>
API (web et future app Flutter)	<code>https://campuslink-api-kn7w.onrender.com</code> — santé : <code>/api/health</code>
Base de données	MongoDB Atlas, cluster M0, base <code>campuslink</code> (fichiers envoyés dans GridFS)
Données	Données de démonstration du 8 octobre 2026 (emploi du temps sur 5 semaines à partir du 5 octobre) : mêmes comptes qu'en section 4.6, mot de passe <code>Campus123!</code> . Relancer le seed pour rafraîchir les dates.
Administrateur	<code>raed.charrad@esprit.tn</code> (mot de passe communiqué séparément)
Vérifié en ligne	Connexion des 4 rôles, pages principales, téléchargement d'un document (GridFS), chat temps réel du covoiturage.

Procédure détaillée et variables : [docs/deploiement.md](#) . Les accès aux comptes Vercel, Render et Atlas sont gérés par Raed.

9. Sécurité : ce que chacun doit savoir

- **Secrets** : `.env` et `.env.local` ne sont jamais commités. Les anciens `SECRET` et `FCM_SERVER_KEY` sont visibles dans l'historique git (commit `12a8462`) : ils doivent être considérés comme compromis et changés.
- **Production** : générer de nouveaux `JWT_SECRET` et clés VAPID ; ne jamais utiliser les comptes de démonstration (`Campus123!`).
- **Mots de passe et codes** : hachés (bcrypt / SHA-256) ; codes à usage unique et liens de réinitialisation limités dans le temps.
- **Limitation des tentatives** : active par défaut (connexion, codes, mots de passe, écritures des modules) ; `RATE_LIMIT_ENABLED=false` seulement en développement.
- **Données personnelles** : profils alumni privés sans consentement, e-mails partagés seulement après un mentorat accepté, coordonnées de covoiturage arrondies pour les non-participants, comparaison de notes à partir de 5 étudiants, données hors ligne effacées à la déconnexion.
- **Journal d'audit** : les actions sensibles sont tracées (consultable par l'administration).
- **Avant chaque fusion** : faire passer `tests/api/security.spec.ts`.

10. Problèmes fréquents et solutions

Symptôme	Solution
L'API s'arrête : « Could not connect to MongoDB »	MongoDB ne tourne pas. Service Windows : <code>Start-Service MongoDB</code> (PowerShell administrateur) ; Docker : <code>docker start campuslink-mongo</code> . Vérifier <code>MONGO_URI</code> .
L'API refuse de démarrer : « Missing required environment variable »	Créer <code>backend/.env</code> depuis <code>.env.example</code> et remplir <code>MONGO_URI</code> et <code>JWT_SECRET</code> .
« Port 4000 / 3000 already in use »	Un serveur tourne déjà : le fermer, ou changer <code>PORT</code> (API) / <code>-p</code> (Next).
« Another next dev server is already running »	Next.js 16 : un seul <code>next dev</code> par dossier. Fermer l'autre terminal.
<code>tsc</code> échoue dans <code>.next/dev/types</code>	Fichiers générés périmés : supprimer <code>next/.next/dev</code> puis relancer.
« Impossible de joindre le serveur » sur le site	L'API n'est pas lancée ou <code>API_URL</code> est faux (<code>next/.env.local</code>).
Je ne reçois pas le code de connexion / le lien de réinitialisation	Sans SMTP, ils sont affichés dans la console de l'API.
Erreur 429 « Trop de tentatives » en développement	Attendre la fin de la fenêtre, ou <code>RATE_LIMIT_ENABLED=false</code> dans <code>backend/.env</code> .
Le hors ligne ou le push ne marchent pas	Le service worker n'existe qu'en build de production (section 5.6) ; le push demande aussi les clés VAPID et l'autorisation du navigateur.
Le chat du covoiturage ne se connecte pas	<code>NEXT_PUBLIC_REALTIME_URL</code> doit pointer vers l'API au moment du build ; l'adresse du site doit être dans <code>CORS_ORIGINS</code> .
Erreur d'hydratation liée à <code>data-dav10</code> dans la console	Extension de navigateur (Office/WebDAV) : sans gravité, déjà neutralisée sur <code><html></code> .
Données de démonstration « dans le passé »	Relancer <code>npm run seed:demo</code> (idempotent) : les données sont recalées sur la date du jour.

11. Documentation de référence

Document	Contenu
<code>README.md</code>	Présentation, démarrage rapide, liens
<code>docs/architecture.md</code>	Architecture, choix techniques, sécurité, modèle de données
<code>docs/deploiement.md</code>	Installation locale, production, nginx / HTTPS, sauvegarde
<code>docs/specifications/</code>	User stories et critères d'acceptation (modules 1, 4, 5, 7, 8, 9, 10)
<code>docs/phase1 2 3-contract.md</code>	Contrats techniques détaillés (API, règles, écrans), en anglais
<code>backend/README.md</code> , <code>backend/docs/*.md</code>	Services internes de l'API, documentation par module (dont la connexion temps réel pour Flutter)
<code>next/README.md</code> , <code>next/docs/*.md</code>	Conventions du site, hors ligne, BFF, documentation par module
<code>tests/README.md</code>	Lancer et comprendre les tests
<code>docs/quality/</code>	Rapports Lighthouse
<code>docs/demo/</code>	Vidéos de démonstration par rôle (phase 1)

Ce guide est généré depuis `docs/guide/guide-equipe.html` (`node docs/guide/build-pdf.js`) : modifiez le HTML puis régénérez le PDF.